

# Nhập môn thuật toán

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

MIT Press, ấn bản thứ ba, 2009

*Nguồn gốc: Introduction to Algorithms, Third Edition*

English Materials / Introduction To Algorithm - 3rd Edition.pdf

## Tóm tắt

Đây là bản ghi chú tiếng Việt mở rộng cho *Introduction to Algorithms*, ấn bản thứ ba. Sách là tài liệu tham khảo kinh điển về thuật toán và cấu trúc dữ liệu, bao phủ nền tảng phân tích, sắp xếp, cấu trúc dữ liệu, kỹ thuật thiết kế, đồ thị, số học, xâu, hình học tính toán, NP-đầy đủ và thuật toán xấp xỉ. Bản này không dịch mã giả thành code template; nó chuyển ngữ cấu trúc, vai trò của từng phần và các điểm kỹ thuật quan trọng để người học định hướng đọc sâu.

## 1 Thông tin sách

|                     |                                                                          |
|---------------------|--------------------------------------------------------------------------|
| <b>Tác giả</b>      | Thomas H. Cormen; Charles E. Leiserson; Ronald L. Rivest; Clifford Stein |
| <b>Nhà xuất bản</b> | The MIT Press                                                            |
| <b>Ấn bản</b>       | Thứ ba                                                                   |
| <b>Năm</b>          | 2009                                                                     |
| <b>ISBN</b>         | 978-0-262-03384-8                                                        |

Sách được dùng rộng rãi trong các khóa thuật toán bậc đại học và sau đại học. Mỗi chương tương đối độc lập, thường gồm mô hình bài toán, mã giả, chứng minh tính đúng, phân tích độ phức tạp và bài tập. Điểm mạnh của sách là mức độ bao quát và sự chặt chẽ; điểm cần lưu ý là mã giả nhằm giải thích ý tưởng, không phải thư viện code sẵn để nộp bài.

## 2 Cách dùng trong thư viện này

Trong luyện OI, CLRS nên được dùng như tài liệu tham khảo nền tảng:

- Đọc để hiểu invariant, proof và phân tích, rồi tự viết code theo ngôn ngữ và chuẩn thi của mình.
- Không dịch trực tiếp mã giả thành template; cần xử lý chỉ số, sentinel, kiểu dữ liệu và biên theo môi trường lập trình cụ thể.
- Với chương nâng cao, ưu tiên nắm mô hình và định lý trước khi học tối ưu cài đặt.

Các chương sát OI nhất là 1–9, 10–17, 21–26, 30–35 và phụ lục toán. Các chương 18–20, 27–29 hữu ích cho nền tảng rộng hơn nhưng ít xuất hiện trực tiếp trong bài thi phổ thông.

### 3 Ghi chú mở rộng theo phần

#### Phần I. Nền tảng

**Chương 1** đặt thuật toán vào bối cảnh tính toán: thuật toán là công nghệ cốt lõi, khác biệt giữa  $O(n^2)$  và  $O(n \log n)$  trở nên rất lớn khi dữ liệu tăng. Chương này cũng nhấn mạnh rằng kiến thức thuật toán giúp lập trình viên giải bài lớn hơn, không chỉ viết code chạy được trên ví dụ nhỏ.

**Chương 2** giới thiệu khung phân tích bằng insertion sort và merge sort. Insertion sort minh họa loop invariant: khởi tạo, duy trì, kết thúc. Merge sort minh họa chia để trị và truy hồi

$$T(n) = 2T(n/2) + \Theta(n).$$

Người học nên chú ý cách tách ba việc: mô tả thuật toán, chứng minh đúng, và phân tích thời gian.

**Chương 3** chuẩn hóa ký hiệu tiệm cận:  $O$ ,  $\Omega$ ,  $\Theta$ ,  $o$ ,  $\omega$ , cùng các hàm chuẩn như logarit, đa thức, mũ và giai thừa. Một điểm hay sai là dùng  $O(\cdot)$  trong biểu thức như một hàm ẩn danh, không phải phép bằng thông thường.

**Chương 4** phát triển chia để trị qua maximum subarray, nhân ma trận Strassen và các phương pháp giải truy hồi: thế, cây đệ quy và master theorem. Đây là chương nền cho rất nhiều thuật toán: merge sort, divide-and-conquer DP, FFT và closest pair.

**Chương 5** trình bày phân tích xác suất và thuật toán ngẫu nhiên qua hiring problem, biến chỉ thị và randomized algorithms. Tuy ngắn, chương này quan trọng vì biến chỉ thị cho phép tính kỳ vọng bằng tuyến tính của kỳ vọng mà không cần độc lập.

#### Phần II. Sắp xếp và thống kê thứ tự

**Chương 6** giới thiệu heap và heapsort. Heap là cây nhị phân gần đầy đủ lưu trong mảng, hỗ trợ EXTRACT-MAX, INSERT, tăng khóa và priority queue trong  $O(\log n)$ . Với OI, heap là cấu trúc cơ bản cho Dijkstra, greedy và mô phỏng sự kiện.

**Chương 7** phân tích quicksort. Worst case là  $\Theta(n^2)$ , nhưng bản randomized có kỳ vọng  $\Theta(n \log n)$  và thường nhanh trong thực tế. Điểm cần nắm là partition invariant và phân tích bằng số lần so sánh giữa cặp phần tử.

**Chương 8** chứng minh cận dưới  $\Omega(n \log n)$  cho comparison sort bằng decision tree, rồi đưa ra các thuật toán tuyến tính khi có giả thiết mạnh hơn: counting sort, radix sort và bucket sort. Trong lập trình thi đấu, counting sort/radix sort hữu ích khi khóa là số nguyên trong miền kiểm soát được.

**Chương 9** trình bày tìm min/max, randomized selection kỳ vọng tuyến tính và deterministic selection worst-case tuyến tính bằng median-of-medians. Đây là cơ sở lý thuyết của việc chọn phần tử thứ  $k$ , dù trong thực tế thường dùng quickselect ngẫu nhiên hoặc thư viện đã tối ưu.

#### Phần III. Cấu trúc dữ liệu

**Chương 10** nhắc stack, queue, linked list, biểu diễn object bằng mảng và cây gốc. Chương này tưởng đơn giản nhưng quan trọng vì nhiều cấu trúc nâng cao chỉ là tổ hợp của pointer, array và invariant.

**Chương 11** trình bày hash table: direct addressing, chaining, hash functions, open addressing và perfect hashing. Người học nên phân biệt kỳ vọng do giả thiết hash với worst-case đối nghịch; trong OI cần lưu ý collision, custom hash và bộ nhớ.

**Chương 12** là cây tìm kiếm nhị phân: inorder walk, search, minimum, successor, insert, delete và phân tích cây ngẫu nhiên. Các thao tác này chuẩn bị cho red-black tree và nhiều balanced BST khác.

**Chương 13** trình bày red-black tree. Invariant màu bảo đảm chiều cao  $O(\log n)$ ; rotate là phép biến đổi cục bộ giữ thứ tự inorder; insert/delete cần sửa màu và xoay để khôi phục invariant. Đây là nền của nhiều container như ordered map/set.

**Chương 14** nói về tăng cường cấu trúc dữ liệu. Ví dụ order-statistic tree thêm kích thước subtree để trả lời select/rank; interval tree thêm max endpoint để tìm đoạn giao. Bài học tổng quát: chỉ thêm thông tin phụ nếu cập nhật được trong thời gian cùng bậc với thao tác gốc.

## Phần IV. Kỹ thuật thiết kế và phân tích nâng cao

**Chương 15** là quy hoạch động. Rod cutting, matrix-chain multiplication, longest common subsequence và optimal BST minh họa hai tính chất: optimal substructure và overlapping subproblems. Cần phân biệt top-down memoization, bottom-up table và cách khôi phục nghiệm bằng bảng lựa chọn.

**Chương 16** là thuật toán tham lam. Activity selection, Huffman coding, matroid và scheduling cho thấy greedy cần chứng minh bằng exchange argument, cut-and-paste hoặc cấu trúc matroid; không chỉ vì “chọn tốt nhất hiện tại”.

**Chương 17** là phân tích khấu hao. Aggregate analysis, accounting method và potential method giải thích vì sao một số thao tác đắt thỉnh thoảng vẫn cho chi phí trung bình tốt, ví dụ dynamic table resize. Đây là nền cho DSU, splay và nhiều cấu trúc online.

## Phần V. Cấu trúc dữ liệu nâng cao

**Chương 18** trình bày B-tree, cấu trúc tối ưu cho bộ nhớ ngoài. Mỗi node chứa nhiều khóa để giảm số lần truy cập disk/page. Insert tách node đầy; delete cần mượn/gộp để giữ số khóa tối thiểu.

**Chương 19** là Fibonacci heap. Cấu trúc này cho DECREASE-KEY khấu hao  $O(1)$ , giúp cải thiện một số thuật toán đồ thị về mặt lý thuyết. Trong thực tế OI, binary heap hoặc pairing heap thường đơn giản hơn, nhưng Fibonacci heap quan trọng để hiểu tradeoff giữa lý thuyết và cài đặt.

**Chương 20** trình bày van Emde Boas tree cho universe số nguyên hữu hạn, hỗ trợ predecessor/successor trong  $O(\log \log U)$ . Chương này liên quan đến cấu trúc theo miền giá trị, bit decomposition và recursion trên universe.

**Chương 21** là disjoint-set union. Hai heuristic union by rank và path compression cho thời gian gần hằng số, chính xác là theo hàm nghịch đảo Ackermann. DSU là cấu trúc xuất hiện trực tiếp trong Kruskal, connectivity offline và nhiều bài toán gom nhóm.

## Phần VI. Thuật toán đồ thị

**Chương 22** trình bày biểu diễn đồ thị, BFS, DFS, topological sort và strongly connected components. BFS cho khoảng cách không trọng số; DFS cho thời gian khám phá/kết thúc, cạnh cây/ngược/xuôi/chéo và nền của SCC. Thuật toán SCC dùng thứ tự kết thúc trên đồ thị gốc rồi DFS trên đồ thị chuyển vị.

**Chương 23** là cây khung nhỏ nhất. Quy tắc cut/cycle dẫn đến Kruskal và Prim. Kruskal dùng DSU, Prim dùng priority queue; cả hai là ví dụ điển hình của greedy có chứng minh bằng cut property.

**Chương 24** là đường đi ngắn nhất một nguồn. Bellman–Ford xử lý cạnh âm và phát hiện chu trình âm; DAG shortest path dùng topo order; Dijkstra cần trọng số không âm; difference constraints chuyển hệ bất đẳng thức thành shortest path.

**Chương 25** là mọi cặp đường đi ngắn nhất. Floyd–Warshall là DP trên tập đỉnh trung gian; Johnson dùng reweighting để xử lý đồ thị thưa có cạnh âm nhưng không có chu trình âm. Phần matrix multiplication cho thấy cách nhìn đại số của shortest path.

**Chương 26** là luồng cực đại. Ford–Fulkerson, Edmonds–Karp, matching hai phía, push-relabel và relabel-to-front đều dựa trên residual network, augmenting path/preflow và max-flow min-cut theorem. Đây là nền của nhiều bài toán chọn, ghép, cắt và ràng buộc.

## Phần VII. Chủ đề chọn lọc

**Chương 27** giới thiệu thuật toán đa luồng, work/span, scheduler và các ví dụ matrix multiplication, merge sort. Với OI cổ điển ít dùng, nhưng hữu ích để hiểu parallel algorithm và giới hạn tăng tốc.

**Chương 28** là phép toán ma trận: giải hệ tuyến tính, nghịch đảo, ma trận xác định dương và least squares. Đây là nền cho tối ưu, đồ họa, ML và một số bài toán đại số tuyến tính.

**Chương 29** trình bày quy hoạch tuyến tính: dạng chuẩn/slack, mô hình hóa bài toán, simplex, đối ngẫu và nghiệm khả thi ban đầu. Nhiều bài toán tối ưu tổ hợp có thể được hiểu qua LP relaxation và duality.

**Chương 30** là đa thức và FFT. DFT/FFT giảm nhân đa thức từ  $O(n^2)$  xuống  $O(n \log n)$  bằng đánh giá tại căn bậc  $n$  của đơn vị rồi nội suy. Trong OI, đây là nền của convolution, nhân số lớn và đếm bằng tích chập.

**Chương 31** là thuật toán số học: gcd, Euclid mở rộng, số học modulo, CRT, lũy thừa nhanh, RSA, kiểm tra nguyên tố và phân tích thừa số. Đây là một trong những phần sát OI nhất.

**Chương 32** là khớp xâu: naive, Rabin–Karp, automata hữu hạn và KMP. Điểm cốt lõi của KMP là hàm tiền tố cho phép tránh so sánh lại; Rabin–Karp dùng rolling hash và cần kiểm soát collision.

**Chương 33** là hình học tính toán: kiểm tra giao đoạn, sweep line cho cặp đoạn giao, convex hull và closest pair. Khi cài đặt cần đặc biệt chú ý sai số số thực, orientation, xử lý điểm thẳng hàng và biên.

**Chương 34** là NP-đầy đủ. Sách định nghĩa P, NP, verifier đa thức, reduction và cách chứng minh NP-complete. Quan trọng nhất là hướng reduction: muốn chứng minh bài mới khó, giảm một bài đã biết khó về bài mới trong thời gian đa thức.

**Chương 35** là thuật toán xấp xỉ. Vertex cover, TSP có bất đẳng thức tam giác, set cover, subset sum và các bài toán khác minh họa cách chấp nhận nghiệm không tối ưu nhưng có bảo đảm tỷ lệ. Đây là cách phản ứng khi bài toán tối ưu chính xác là NP-hard.

## Phần VIII. Phụ lục toán học

Phụ lục cung cấp công cụ nền: tổng và tích, tập hợp, quan hệ, hàm, đồ thị, cây, đếm, xác suất rời rạc và ma trận. Với người học OI, phụ lục không nên bỏ qua: nhiều lỗi trong chứng minh thuật toán đến từ việc dùng sai tổng, xác suất hoặc quy nạp.

## 4 Phụ lục kỹ thuật ưu tiên cho OI

Phần này gom lại những điểm kỹ thuật của CLRS thường trở thành “xương sống” khi giải và trình bày bài OI. Đây không phải bản thay thế cho từng chứng minh trong sách, nhưng giúp bản dịch bớt chỉ là mục lục có chú thích: mỗi mục nêu invariant, công thức hoặc điều kiện cần kiểm tra khi chuyển ý tưởng thành lời giải thi đấu.

### 4.1 Bất biến vòng lặp và chứng minh đúng

Khuôn chứng minh ở Chương 2 xuất hiện xuyên suốt sách. Với một vòng lặp, cần viết rõ mệnh đề bất biến  $I$  và kiểm tra ba bước:

- **Khởi tạo:**  $I$  đúng trước lần lặp đầu tiên.
- **Duy trì:** nếu  $I$  đúng ở đầu một lần lặp thì sau thân vòng lặp,  $I$  vẫn đúng cho lần kế tiếp.
- **Kết thúc:** khi điều kiện lặp sai,  $I$  cộng với điều kiện dừng suy ra kết quả mong muốn.

Trong OI, bất biến không nhất thiết phải dài. Ví dụ với insertion sort, sau khi xử lý vị trí  $j$ , đoạn đầu mảng đã được sắp và chứa đúng các phần tử ban đầu của đoạn đó. Với BFS, bất biến là hàng đợi chứa biên của những đỉnh đã biết khoảng cách nhưng chưa quét hết cạnh. Với Dijkstra, bất biến mạnh hơn: mỗi đỉnh đã lấy ra khỏi heap có khoảng cách tối ưu cuối cùng, điều này chỉ đúng khi mọi cạnh có trọng số không âm.

## 4.2 Truy hồi và master theorem

Với chia để trị, bước đầu tiên là viết đúng truy hồi. Dạng chuẩn của master theorem là

$$T(n) = aT(n/b) + f(n),$$

trong đó  $a$  là số bài con, mỗi bài con có kích thước  $n/b$ , còn  $f(n)$  là chi phí chia và ghép. So sánh  $f(n)$  với  $n^{\log_b a}$ :

- nếu  $f(n) = O(n^{\log_b a - \varepsilon})$  thì  $T(n) = \Theta(n^{\log_b a})$ ;
- nếu  $f(n) = \Theta(n^{\log_b a} \log^k n)$  thì  $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ ;
- nếu  $f(n) = \Omega(n^{\log_b a + \varepsilon})$  và điều kiện regularity đúng, nghiệm là  $\Theta(f(n))$ .

Khi bài thi có kích thước không chia đều hoặc có nhiều nhánh khác nhau, đừng ép vào master theorem quá sớm. Cây đệ quy, thể quy nạp, Akra–Bazzi hoặc phân tích theo tổng kích thước các đoạn thường an toàn hơn. Ví dụ merge sort dùng tổng chi phí mỗi tầng là  $\Theta(n)$ ; closest pair cũng cần chứng minh rằng bước ghép chỉ xét số điểm hằng số trong dải, nếu không truy hồi đúng sẽ bị phá vỡ.

## 4.3 Xác suất và biến chỉ thị

Chương 5 dùng biến chỉ thị để biến một đại lượng khó thành tổng các biến Bernoulli. Nếu  $X = \sum_i X_i$  thì

$$\mathbf{E}[X] = \sum_i \mathbf{E}[X_i]$$

luôn đúng, không cần các  $X_i$  độc lập. Đây là kỹ thuật chính trong phân tích quicksort ngẫu nhiên, hashing và nhiều thuật toán randomized.

Khi áp dụng trong lời giải, cần phân biệt ba tầng:

- **kỳ vọng theo random của thuật toán**, ví dụ pivot quicksort;
- **kỳ vọng theo input ngẫu nhiên**, thường không đủ nếu đề có dữ liệu đối nghịch;
- **xác suất lỗi**, như rolling hash, Miller–Rabin hoặc randomized contraction, cần nêu cách giảm rủi ro bằng chọn modulo/base hoặc lặp.

## 4.4 Sắp xếp, selection và cận dưới

Cận dưới decision tree ở Chương 8 nói rằng mọi thuật toán so sánh tổng quát cần  $\Omega(n \log n)$  phép so sánh trong worst case. Nếu một lời giải tuyên bố sắp xếp nhanh hơn, nó phải dùng thêm giả thiết: khóa nguyên trong miền nhỏ, chữ số có số lượng cố định, bucket có phân bố tốt, hoặc cần ít hơn toàn bộ thứ tự.

Selection có hai hướng đọc:

- randomized quickselect thường đơn giản và nhanh, kỳ vọng  $O(n)$ ;
- median-of-medians bảo đảm worst-case  $O(n)$ , nhưng hằng số lớn và chủ yếu dùng để hiểu lý thuyết hoặc đối phó dữ liệu xấu.

Với bài cần phần tử thứ  $k$ , nên kiểm tra xem có thật sự phải sắp toàn bộ hay chỉ cần partition, heap kích thước  $k$ , binary search trên đáp án, hoặc selection trên miền giá trị.

## 4.5 Hash table và cây tìm kiếm

Hash table trong CLRS tách rõ mô hình trung bình và worst case. Chaining dễ cài và chịu tải lớn; open addressing tiết kiệm con trỏ nhưng cần chính sách probe và ngưỡng tải cẩn thận. Trong thi đấu, lỗi thường gặp là dùng hash của thư viện trên input đối nghịch, hoặc quên rằng xác suất collision của rolling hash không phải bằng không.

Cây tìm kiếm nhị phân ở Chương 12–14 cho ba ý chính:

- thứ tự inorder là tài sản cần giữ khi xoay hoặc cập nhật;
- cân bằng chiều cao biến thao tác từ tuyến tính thành  $O(\log n)$ ;
- augmentation chỉ hợp lệ nếu thông tin phụ của một node tính được từ node đó và số hằng các con sau mỗi phép xoay.

Order-statistic tree là mẫu chuẩn: lưu kích thước subtree để hỗ trợ SELECT và RANK. Từ mẫu này có thể suy ra nhiều biến thể như đếm số phần tử nhỏ hơn  $x$ , tìm phần tử thứ  $k$  trong cửa sổ, hoặc duy trì tổng/maximum trên cây cân bằng.

## 4.6 Quy hoạch động

Một DP tốt cần xác định đủ bốn thành phần:

- **trạng thái**:  $dp[\dots]$  biểu diễn đúng đại lượng tối ưu hoặc số cách nào;
- **chuyển**: công thức lấy từ lựa chọn cuối, điểm chia, hoặc phần tử được thêm vào;
- **thứ tự tính**: bảo đảm mọi trạng thái phụ đã sẵn sàng;
- **khôi phục nghiệm**: nếu đề cần phương án, lưu quyết định hoặc truy ngược từ bảng.

Các ví dụ CLRS là mẫu kinh điển. Matrix-chain multiplication dùng

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}.$$

LCS dùng

$$c[i, j] = \begin{cases} c[i - 1, j - 1] + 1, & x_i = y_j, \\ \max(c[i - 1, j], c[i, j - 1]), & x_i \neq y_j. \end{cases}$$

Optimal BST minh họa DP có chi phí xác suất và tổng trọng số đoạn. Khi chuyển sang OI, các ví dụ này dẫn đến tối ưu Knuth, chia để trị, monotone queue, convex hull trick hoặc bitmask DP, nhưng mọi tối ưu đều phải giữ nguyên nghĩa trạng thái gốc.

## 4.7 Tham lam

CLRS nhấn mạnh rằng greedy phải có chứng minh. Ba mẫu chứng minh thường dùng:

- **exchange argument**: biến một nghiệm tối ưu bất kỳ thành nghiệm có lựa chọn greedy mà không làm xấu đáp án;
- **cut property**: trong MST, cạnh nhẹ nhất băng qua một cut an toàn để thêm vào cây;
- **matroid**: nếu hệ độc lập thỏa tính di truyền và trao đổi, greedy theo trọng số hoạt động.

Activity selection là ví dụ chọn kết thúc sớm nhất; Huffman coding là ví dụ ghép hai tần suất nhỏ nhất. Với bài mới, nếu không tìm được một trong các mẫu trên, greedy có thể chỉ là heuristic. Khi nghi ngờ, hãy thử dựng phản ví dụ nhỏ trước khi viết lời giải.

## 4.8 Phân tích khấu hao

Khấu hao không phải trung bình trên input; nó là bảo đảm trên mọi dãy thao tác. Ba cách nhìn tương đương:

- **aggregate**: tổng chi phí của  $m$  thao tác rồi chia cho  $m$ ;
- **accounting**: thao tác rẻ trả thêm tiền để bù thao tác đắt sau này;
- **potential**: định nghĩa thế năng  $\Phi(D)$  của cấu trúc, chi phí khấu hao là  $c_i + \Phi(D_i) - \Phi(D_{i-1})$ .

Dynamic table là ví dụ chuẩn: resize tốn  $O(n)$ , nhưng nếu nhân đôi khi đầy và co lại thận trọng, mỗi phần tử chỉ bị copy số lần hữu hạn theo cấp số nhân. Trong DSU, path compression cũng cần phân tích khấu hao tinh vi hơn; kết quả gần hằng số không có nghĩa là mỗi lệnh riêng lẻ luôn hằng số.

## 4.9 Disjoint-set union

DSU có ba thao tác: MAKE-SET, FIND-SET, UNION. Hai heuristic quan trọng là union by rank/size và path compression. Sau khi gọi FIND-SET( $x$ ), mọi đỉnh trên đường từ  $x$  về root có thể trở thẳng về root, làm phẳng cây cho các lần sau.

Ứng dụng OI thường gặp:

- Kruskal và các biến thể MST offline;
- connectivity offline khi chỉ thêm cạnh hoặc xử lý ngược thao tác xóa;
- gom nhóm ràng buộc bằng parity/potential DSU;
- DSU on tree là kỹ thuật khác tên, không phải cấu trúc union-find chuẩn.

Khi DSU có trọng số hiệu hoặc parity, bất biến không chỉ là root, mà còn là giá trị tương đối từ node lên cha. Nén đường phải cập nhật giá trị tương đối cùng lúc với cập nhật cha.

## 4.10 BFS, DFS và cấu trúc đồ thị

BFS trên đồ thị không trọng số cho khoảng cách theo số cạnh. Nếu có cạnh trọng số 0/1, dùng deque cho 0-1 BFS; nếu trọng số tổng quát không âm, chuyển sang Dijkstra. Lỗi hay gặp là dùng BFS khi trọng số không đồng nhất.

DFS sinh hai mốc thời gian  $d[u]$  và  $f[u]$ . Chúng hỗ trợ:

- phát hiện cạnh ngược và chu trình trong đồ thị có hướng;
- topological sort bằng thứ tự giảm  $f[u]$ ;
- SCC bằng Kosaraju hoặc Tarjan;

- cầu, khớp, biconnected components bằng low-link.

CLRS trình bày SCC qua DFS trên đồ thị chuyển vị. Với cài đặt thi đấu, thuật toán Tarjan một DFS thường tiện hơn, nhưng invariant low-link vẫn dựa trên cùng ý tưởng về thứ tự khám phá và cạnh quay lại.

#### 4.11 Cây khung nhỏ nhất

Hai định lý an toàn của MST cần nhớ:

- **cut property**: cạnh nhẹ nhất băng qua một cut tôn trọng tập cạnh đã chọn là cạnh an toàn;
- **cycle property**: cạnh nặng nhất trên một chu trình không bắt buộc nằm trong MST.

Kruskal sắp cạnh rồi dùng DSU, tốt cho đồ thị thưa và dễ kiểm soát. Prim mở rộng từ một cây bằng heap, tự nhiên trên đồ thị đặc hoặc khi có adjacency list. Nếu đề yêu cầu MST thứ hai, MST động, bottleneck spanning tree hoặc kiểm tra tính duy nhất, cần lưu thêm thông tin trên đường trong cây, thường bằng LCA hoặc HLD.

#### 4.12 Đường đi ngắn nhất

Mọi thuật toán shortest path trong Chương 24–25 xoay quanh phép relaxation:

nếu  $d[v] > d[u] + w(u, v)$  thì cập nhật  $d[v]$ .

Khác biệt nằm ở thứ tự relaxation:

- Bellman–Ford lặp  $n - 1$  vòng, xử lý cạnh âm và phát hiện chu trình âm;
- DAG shortest path theo topo order, đúng kể cả có cạnh âm nếu không có chu trình;
- Dijkstra dùng đỉnh có  $d$  nhỏ nhất hiện tại, chỉ đúng với trọng số không âm;
- Floyd–Warshall là DP theo tập đỉnh trung gian, phù hợp  $n$  nhỏ;
- Johnson dùng potential từ Bellman–Ford để reweight cạnh, rồi chạy Dijkstra từ từng đỉnh trên đồ thị thưa.

Difference constraints chuyển hệ  $x_j - x_i \leq b_k$  thành cạnh  $i \rightarrow j$  trọng số  $b_k$ . Nếu có chu trình âm, hệ vô nghiệm. Đây là cầu nối giữa bất đẳng thức tuyến tính đơn giản và shortest path.

#### 4.13 Luồng cực đại và matching

Các thuật toán flow dùng residual network. Nếu còn đường tăng từ  $s$  đến  $t$ , tăng luồng trên đường đó; khi không còn, tập đỉnh còn tới được từ  $s$  trong mạng dư tạo ra min cut. Định lý max-flow min-cut nói giá trị max flow bằng dung lượng min cut.

Trong lời giải OI, phần khó thường là mô hình hóa:

- bài chọn/loại với ràng buộc đóng thường thành min cut;
- matching hai phía là max flow trên mạng phân lớp  $s$ -trái-phải- $t$ ;
- vertex capacity cần tách đỉnh thành  $v_{\text{in}} \rightarrow v_{\text{out}}$ ;
- lower/upper bound flow cần xử lý cân bằng nhu cầu trước khi chạy max flow phụ.

Edmonds–Karp dễ chứng minh nhưng chậm; Dinic thường là lựa chọn thực tế trong thi đấu dù không phải thuật toán chính của CLRS. Khi dùng Dinic, vẫn nên trình bày bằng ngôn ngữ residual network, augmenting/blocking flow và cut.



#### 4.14 FFT và đa thức

DFT biến hệ số đa thức thành giá trị tại các căn đơn vị. Nhân đa thức đi theo ba bước:

1. bổ sung hệ số không để độ dài đủ lớn;
2. FFT hai đa thức sang miền giá trị, nhân từng điểm;
3. inverse FFT để trở lại hệ số.

Độ dài thường chọn là lũy thừa của 2 không nhỏ hơn tổng bậc cộng 1. Với số nguyên lớn, cần chú ý sai số floating point; với modulo thân thiện, dùng NTT. Trong bài đếm, convolution thường xuất hiện khi ghép số lượng theo tổng, khoảng cách, màu hoặc bậc.

#### 4.15 Số học

Euclid mở rộng trả về  $x, y$  sao cho

$$ax + by = \gcd(a, b).$$

Từ đó suy ra nghịch đảo modulo khi  $\gcd(a, m) = 1$ , giải phương trình đồng dư tuyến tính và CRT. Lũy thừa nhanh dựa trên khai triển nhị phân của số mũ; cùng ý tưởng dùng cho ma trận, permutation và hàm hợp.

Khi đề có modulo không nguyên tố, không được chia bằng nghịch đảo nếu mẫu không nguyên tố cùng nhau với modulo. Khi dùng CRT, cần phân biệt hệ modulo đôi một nguyên tố cùng nhau với CRT tổng quát. Khi kiểm tra nguyên tố, Miller–Rabin có bộ cơ sở xác định cho 64-bit; phân tích thừa số lớn thường cần Pollard rho, không chỉ trial division.

#### 4.16 Xâu

KMP xây dựng hàm tiền tố  $\pi[i]$ : độ dài biên dài nhất của prefix kết thúc tại  $i$ . Khi mismatch, mẫu không lùi về 0 ngay mà lùi theo biên kế tiếp. Điều này tạo thời gian tuyến tính vì mỗi ký tự chỉ làm con trỏ tiến/lùi hữu hạn lần.

Rabin–Karp dùng hash cho so khớp nhiều vị trí hoặc nhiều mẫu, nhưng cần xác nhận collision nếu bài đòi đúng tuyệt đối. Finite automaton matching cho thấy cùng một ý tưởng có thể viết thành automaton trạng thái; AC automaton là mở rộng tự nhiên cho nhiều mẫu, dù không nằm ở phần chính của CLRS.

#### 4.17 Hình học tính toán

Hạt nhân của hình học phẳng là tích có hướng:

$$\text{cross}(b - a, c - a).$$

Dấu của biểu thức cho biết  $c$  nằm bên trái, bên phải hay thẳng hàng với vector  $a \rightarrow b$ . Từ đó xây dựng kiểm tra giao đoạn, convex hull, rotating calipers và nhiều bài toán diện tích.

Khi cài đặt, phải chọn mô hình số:

- tọa độ nguyên nhỏ: dùng số nguyên 64-bit hoặc 128-bit cho tích;
- tọa độ thực: dùng epsilon nhất quán, tránh so sánh bằng trực tiếp;
- biên thẳng hàng: quyết định có lấy điểm trên cạnh, điểm trùng, đoạn suy biến hay không theo đúng đề.

Closest pair là ví dụ chia để trị có bước ghép tinh tế: sau khi sắp theo  $y$ , mỗi điểm trong dải chỉ cần so với số hàng điểm kế tiếp.

#### 4.18 NP-đầy đủ và xấp xỉ

Để chứng minh bài  $B$  NP-hard, cần lấy một bài  $A$  đã biết NP-hard và giảm  $A \leq_p B$ . Hướng ngược lại là lỗi rất phổ biến. Để chứng minh  $B$  thuộc NP, cần chỉ ra certificate có kích thước đa thức và verifier đa thức.

Với bài tối ưu NP-hard, có ba phản ứng thường gặp trong OI:

- ràng buộc nhỏ cho phép exponential/bitmask/meet-in-the-middle;
- cấu trúc đặc biệt của input làm bài trở nên đa thức;
- đề yêu cầu xấp xỉ, heuristic hoặc subtask.

Chương 35 cung cấp ngôn ngữ về tỷ lệ xấp xỉ. Ví dụ vertex cover có thuật toán 2-approx bằng cách lấy cả hai đầu của một matching tham lam; set cover có phân tích harmonic bằng cách tính chi phí trên từng phần tử được phủ.

### 5 Phụ lục kiểm chứng lời giải theo tinh thần CLRS

CLRS không chỉ là danh sách thuật toán; điểm mạnh của sách là cách biến một ý tưởng thành mệnh đề đúng, có mô hình, có chứng minh và có cận thời gian. Phần này gom các mẫu kiểm chứng thường cần khi viết lời giải OI hoặc editorial. Nó không chuyển mã giả thành template, mà giúp người đọc biết cần chứng minh điều gì trước khi tin một thuật toán.

#### 5.1 Từ bài toán sang mô hình

Trước khi chọn thuật toán, cần xác định đúng đối tượng toán học:

- phần tử cần sắp, chọn, ghép hoặc phủ là gì;
- quan hệ giữa chúng là thứ tự, đồ thị, luồng, ràng buộc tuyến tính, recurrence hay xác suất;
- kích thước  $n, m, U, W$  nào thực sự chi phối độ phức tạp;
- đầu vào có tính chất đặc biệt không: DAG, trọng số không âm, bipartite, khóa nguyên nhỏ, ma trận thưa, hay modulo nguyên tố.

Sai mô hình thường dẫn đến thuật toán nhìn đúng nhưng hỏng ở biên. Ví dụ dùng BFS cho đồ thị có trọng số là nhầm mô hình khoảng cách; dùng Dijkstra khi có cạnh âm là vi phạm điều kiện bất biến; dùng comparison sort rồi kỳ vọng tuyến tính là bỏ qua cận dưới decision tree. Một lời giải tốt nên nêu rõ giả thiết nào của CLRS đang được dùng.

#### 5.2 Chứng minh đúng theo bốn mẫu

Nhiều chứng minh trong CLRS có thể quy về bốn mẫu sau:

- **Bất biến:** sau mỗi bước, cấu trúc trung gian vẫn mang ý nghĩa xác định, như heap property, tập đỉnh đã chốt của Dijkstra, hay bảng DP đã tính xong một miền trạng thái.
- **Quy nạp:** nghiệm kích thước nhỏ đúng, bước chuyển tạo nghiệm kích thước lớn đúng; thường dùng cho DP, chia để trị và cấu trúc cây.

- **Trao đổi:** biến một nghiệm tối ưu bất kỳ thành nghiệm chứa lựa chọn của thuật toán mà không làm xấu đáp án; đây là lõi của greedy và MST.
- **Đối ngẫu/chứng nhận:** đưa ra một vật chứng chặn trên/dưới khớp với nghiệm, như min cut cho max flow, cây cha cho shortest path, hoặc reduction cho NP-hardness.

Khi trình bày, tránh câu “dễ thấy đúng”. Hãy nói rõ mệnh đề đang giữ, vì sao mỗi thao tác không phá mệnh đề đó, và vì sao điều kiện dừng biến mệnh đề thành kết quả cần chứng minh.

### 5.3 Phân tích độ phức tạp

CLRS tách phân tích khỏi cài đặt. Với mỗi thuật toán, cần nêu:

- số lần mỗi vòng lặp hoặc thao tác chính được gọi;
- chi phí của cấu trúc dữ liệu dùng trong thao tác đó;
- bộ nhớ phụ và kích thước bảng/trạng thái;
- worst case, average case hay expected case đang được khẳng định.

Một lỗi phổ biến là nhân nhầm các tầng lặp. Trong DFS, tổng số cạnh quét là  $O(m)$ , không phải  $O(nm)$ , vì mỗi cạnh chỉ được duyệt theo danh sách kề. Trong Dijkstra, cận phụ thuộc số lần EXTRACT-MIN và DECREASE-KEY, nên binary heap,  $d$ -heap và Fibonacci heap cho cận khác nhau. Trong DP trên đoạn, số trạng thái  $O(n^2)$  và số điểm chia  $O(n)$  tạo  $O(n^3)$  nếu không có tối ưu bổ sung.

### 5.4 Kiểm tra điều kiện áp dụng thuật toán

Nhiều thuật toán CLRS đúng chỉ dưới điều kiện cụ thể:

| Thuật toán/kỹ thuật | Điều kiện phải kiểm tra                                            |
|---------------------|--------------------------------------------------------------------|
| Counting/radix sort | khóa nguyên, miền hoặc số chữ số được kiểm soát                    |
| Dijkstra            | mọi trọng số cạnh không âm sau khi reweight nếu có                 |
| Bellman–Ford        | cần phát hiện và xử lý chu trình âm đạt được                       |
| Topological DP      | đồ thị phải là DAG theo hướng chuyển trạng thái                    |
| Greedy interval     | tiêu chí chọn phải có chứng minh trao đổi                          |
| Kruskal/Prim        | trọng số cạnh xác định, đồ thị vô hướng; xử lý rời rạc nếu cần     |
| Max flow            | bảo toàn luồng, dung lượng, mạng dư và cạnh ngược đúng             |
| FFT/NTT             | độ dài đệm đủ, modulo/căn đơn vị hợp lệ hoặc sai số được kiểm soát |
| Hash randomized     | xác suất collision hoặc adversarial input được xét                 |

Bảng này không thay thế chứng minh, nhưng là bộ lọc nhanh. Nếu một điều kiện không đúng, hoặc phải đổi mô hình, hoặc phải thêm bước biến đổi như Johnson reweighting, tách đỉnh cho capacity, thêm nguồn giả, hay xử lý thành phần liên thông riêng.

### 5.5 Từ mã giả CLRS sang cài đặt thi đấu

Mã giả CLRS ưu tiên rõ ý tưởng hơn chi tiết môi trường. Khi cài đặt lại, cần quyết định các điểm sau:

- chỉ số bắt đầu từ 0 hay 1, đoạn đóng hay nửa mở;
- giá trị vô cực có đủ lớn và không tràn khi cộng không;

- so sánh ổn định với số thực, modulo hoặc khóa bằng nhau;
- cấu trúc dữ liệu có hỗ trợ đúng thao tác mà phân tích giả định không;
- recursion depth có vượt stack không, có cần chuyển sang iterative không;
- input lớn có cần đọc nhanh, nén tọa độ hoặc tiết kiệm bộ nhớ không.

Ví dụ, pseudocode Dijkstra có thể dùng `DECREASE-KEY`; trong C++ thường dùng priority queue không xóa khóa cũ mà bỏ qua entry lỗi thời khi pop. Phân tích và bất biến vẫn đúng nếu ta chứng minh rằng entry được xử lý chỉ khi nó khớp với khoảng cách hiện tại. Đây là chuyển đổi cài đặt, không phải thay đổi thuật toán.

## 5.6 Stress test và phản ví dụ

CLRS dạy chứng minh, nhưng khi viết code vẫn cần kiểm thử có hệ thống. Một vòng stress test tốt gồm:

- sinh input nhỏ ngẫu nhiên và so với brute force;
- sinh biên: rỗng, một phần tử, toàn bằng nhau, đã sắp, ngược thứ tự, đồ thị rời rạc, trọng số 0, trọng số âm nếu được phép;
- sinh adversarial case: hash collision giả lập, pivot xấu, heap nhiều khóa trùng, flow có nhiều cạnh ngược;
- kiểm tra invariant trong debug build, ví dụ heap property, tổng luồng vào/ra, thứ tự topo, hay bảng DP không dùng trạng thái chưa tính.

Phản ví dụ nhỏ có giá trị hơn test lớn khó hiểu. Nếu greedy sai, thường có input 3–6 phần tử chứng minh lựa chọn cục bộ bị kẹt. Nếu DP sai, thường có trạng thái nhỏ cho thấy thiếu thông tin trong định nghĩa  $dp$ . Nếu thuật toán đồ thị sai, hãy thử đồ thị có nhiều thành phần, cạnh song song, self-loop, trọng số bằng nhau và đường đi thay thế.

## 5.7 Viết editorial theo chuẩn kỹ thuật

Một editorial chịu ảnh hưởng CLRS nên có cấu trúc:

1. mô hình hóa bài toán và ký hiệu;
2. nhận xét then chốt hoặc định lý nhỏ;
3. thuật toán ở mức ý tưởng, không lệ thuộc ngôn ngữ;
4. chứng minh đúng bằng bất biến/quy nạp/trao đổi/chứng nhận;
5. phân tích thời gian và bộ nhớ;
6. chi tiết cài đặt rủi ro: indexing, overflow, data structure, biên.

Nếu lời giải dùng kết quả nổi tiếng như max-flow min-cut, cut property của MST, master theorem, hoặc cận dưới comparison sort, nên gọi tên kết quả và nêu vì sao điều kiện của nó đúng trong bài. Đây là cách biến kiến thức từ CLRS thành lời giải có thể kiểm tra, thay vì chỉ nêu tên thuật toán.

## 6 Bản đồ ưu tiên cho OI

- **Bắt buộc:** 1–9, 10–17, 21–26, 30–35.

- **Nên biết:** 18, 19, 20 nếu học cấu trúc dữ liệu nâng cao hoặc bộ nhớ ngoài.
- **Theo hướng chuyên sâu:** 27–29 cho song song, đại số tuyến tính và tối ưu tuyến tính.
- **Luôn đối chiếu:** phụ lục về tổng, xác suất, đồ thị và ma trận.

## 7 Ghi chú về trích xuất

Tệp PDF có 1313 trang và lớp văn bản trích xuất tốt. Một số ký hiệu trong mục lục, như dấu đánh sao cho phần nâng cao, có thể hiện thành ký tự lạ khi dùng `pdftotext`. Bản dịch đầy đủ từng chương cần đối chiếu PDF gốc khi chuyển công thức, hình, bảng và mã giả sang LaTeX. Mã giả nên giữ là mã giả tham khảo, không chuyển thành code template.

## 8 Tình trạng bản dịch

Bản hiện tại là ghi chú dịch mở rộng và hướng dẫn chương, không phải bản dịch từng trang của sách. So với bản ghi chú ban đầu, nó đã bổ sung định hướng sử dụng, ghi chú kỹ thuật cho toàn bộ 35 chương, bản đồ ưu tiên cho OI và cảnh báo về việc không dịch mã giả thành template.